

Практическая работа «Развёртывание смарт-контракта в тестовой сети ganache с frontend-частью на Node.js»

Цель работы:

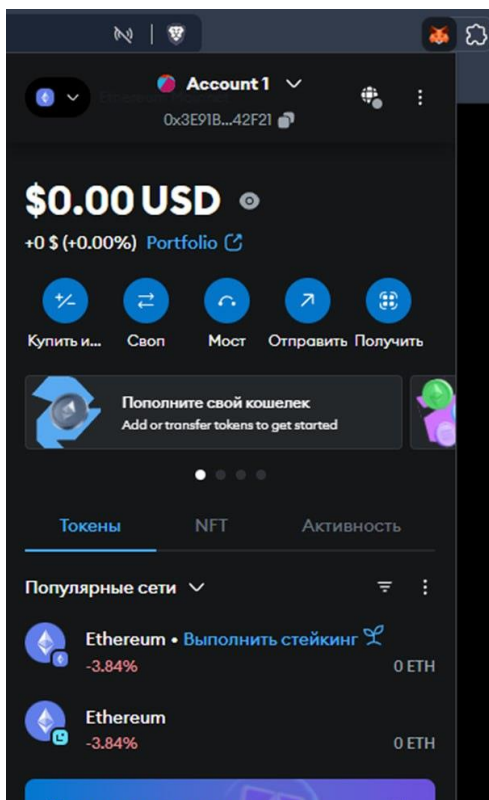
Целью практической работы является получение навыков разработки, развертывания, настройки и взаимодействия со смарт-контрактами в тестовой блокчейн-среде с использованием локальной сети Ganache.

Ход работы:

На первом этапе необходимо создать кошелек Metamask и установить расширение для веб-браузера.

MetaMask — это дополнение (плагин, расширение), выполняющее роль криптокошелька и интерфейса для взаимодействия с блокчейн-сетями. С его помощью пользователи могут подписывать транзакции, использовать к API dApps и работать как с реальной, так и с тестовой сетью Ethereum (включая Ganache).

Зарегистрироваться нужно по ссылке: <https://metamask.io>. После создания кошелька расширение будет выглядеть так:



Далее потребуется установить Git Bash (все дальнейшие операции с консолью будут выполняться именно в Git Bash), Node.js, Truffle и Ganache.

1. По этой ссылке устанавливается Git Bash: <https://git-scm.com/downloads/win>

winget install --id Git.Git -e --source winget

Git Bash — это консольное приложение для Windows, предоставляющее интерфейс командной строки в стиле Unix. Оно позволяет использовать команды Git, а также базовые команды Linux, что удобно при работе с системами контроля версий и при разработке на Node.js.

2. Для установки Node.js: <https://nodejs.org/en/download> (необходимо скачать версию 18.20.2, для проверки версии используется команда **node-v**).

winget install Schniz.fnm

fnm install 18

Node.js — это среда выполнения JavaScript вне браузера, основанная на движке V8. Она позволяет запускать JavaScript на серверной стороне, использовать модули, работать с файлами, сетью и API. Node.js часто применяется в разработке серверов, скриптов и CLI-инструментов.

3. При помощи npm необходимо установить truffle и ganache:

npm install -g truffle

```
zxc@Zxcnerma MINGW64 ~/Desktop/Smart
$ npm install -g truffle
npm WARN deprecated testrpc@0.0.1: testrpc has been renamed to ganache-cli, please use this package from now on.
npm WARN deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache if you want a good and tested way to coalesce async requests by a key value, which is much more comprehensive and powerful.
npm WARN deprecated @truffle/source-map-utils@1.3.119: Package no longer supported. Contact Support at https://www.npmjs.com/support for more info.
npm WARN deprecated mkdirp-promise@5.0.1: This package is broken and no longer maintained. 'mkdirp' itself supports promises now, please switch to that.
npm WARN deprecated rimraf@2.7.1: Rimraf versions prior to v4 are no longer supported
npm WARN deprecated level-concat-iterator@3.1.0: Superseded by abstract-level (https://github.com/Level/community#faq)
npm WARN deprecated @truffle/db-loader@0.2.36: Package no longer supported. Contact Support at https://www.npmjs.com/support for more info.
```

Truffle — это фреймворк для разработки, тестирования и развертывания смарт-контрактов. Он тесно интегрируется с Ganache и позволяет легко управлять миграциями (развёртыванием) и тестированием контрактов на

локальной или тестовой сети.

npm install -g ganache

```
zxc@Zxcnerma MINGW64 ~/Desktop/Smart
● $ npm install -g ganache

added 336 packages in 6s

6 packages are looking for funding
run `npm fund` for details
```

Ganache — это локальная блокчейн-среда, предназначенная для разработки и тестирования смарт-контрактов. Она позволяет запускать тестовую сеть Ethereum на локальном компьютере, быстро проводить транзакции и использовать подробные логи. Ganache обеспечивает разработчикам полный контроль над сетью и учетными записями.

После установки необходимо создать папку проекта (в примере это папка “Smart”) и инициализировать его при помощи **Truffle**:

truffle init

```
zxc@Zxcnerma MINGW64 ~/Desktop/Smart
$ truffle init

Starting init...
=====

> Copying project files to C:\Users\zxc\Desktop\Smart

Init successful, sweet!

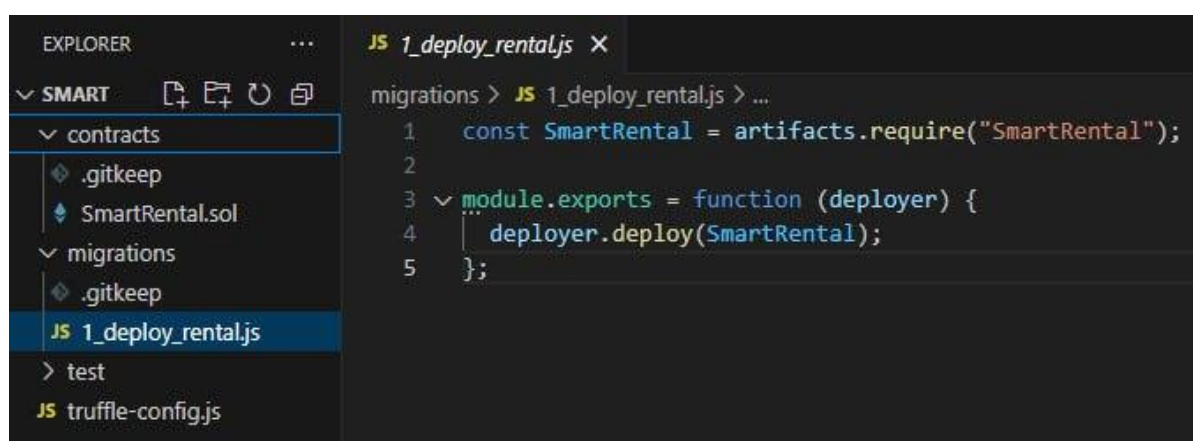
Try our scaffold commands to get started:
  $ truffle create contract YourContractName # scaffold a contract
  $ truffle create test YourTestName         # scaffold a test

http://trufflesuite.com/docs
```

Далее необходимо конфигурировать файл проекта **truffle-config.js** (ввести необходимые параметры) для развертывания контракта:

```
JS truffle-config.js ●
JS truffle-config.js > ...
1  module.exports = {
2    networks: {
3      development: {
4        host: "127.0.0.1",
5        port: 8545,
6        network_id: "*"
7      }
8    },
9    compilers: {
10     solc: {
11       version: "0.8.20"
12     }
13   }
14 };
```

В папке **migrations** создается файл **1_deploy_rental.js** (ВАЖНО назвать файл с припиской 1_, т.к. без нее контракт не развернется. Truffle выполняет миграции в порядке чисел в названии файлов (1_, 2_ и т.д.).)



```
EXPLORER
  SMART
    contracts
      .gitkeep
      SmartRental.sol
    migrations
      .gitkeep
      JS 1_deploy_rental.js
    test
    JS truffle-config.js

JS 1_deploy_rental.js X
migrations > JS 1_deploy_rental.js > ...
1  const SmartRental = artifacts.require("SmartRental");
2
3  module.exports = function (deployer) {
4    deployer.deploy(SmartRental);
5  };
```

Для поднятия тестовой сети необходимо ввести команду **ganache** в КОНСОЛЬ.


```

zxc@Zxcnerma MINGW64 ~/Desktop/Smart
$ ganache
ganache v7.9.2 (@ganache/cli: 0.10.2, @ganache/core: 0.10.2)
Starting RPC server

Available Accounts
=====
(0) 0x710a170e6E6Bb72F915282DDb991d257D9F1102D (1000 ETH)
(1) 0x8426e3351279E03DD078F35Be8875985E77294dE (1000 ETH)
(2) 0x2f661f250D6291d751E251e8505428b4594FD601 (1000 ETH)
(3) 0x39cbb20E48A1e8291a4fD21b323aA2DfCA0da71B (1000 ETH)
(4) 0x1b26F89aB18B96Bb821093aA71d7f5c2999Ce520 (1000 ETH)
(5) 0x3A7dFA04B01ced14Af990850b1b0B9F4B135Bb23 (1000 ETH)
(6) 0x38526d64D9714Dc598524790eA17AcdBf4ACFEA5 (1000 ETH)
(7) 0x5E3A3A023a467c53068f67254A9BfaB0b4Fa2c65 (1000 ETH)
(8) 0x4AB64d8C43bF1d9947188bad0050c661882AE453 (1000 ETH)
(9) 0x15Ca687a5700056846E75a8c5693625667757f5f (1000 ETH)

Private Keys
=====
(0) 0xeacaf13feeb08ee7c827aaa373707ff1e22aeed1ef89f24393f7af504d4149d9
(1) 0xc37df726e46915263ea7469b913792edde787666d83a9780df64c50ca993dc36
(2) 0x6cca43aa8c9fb05590c0799f972df69153bae18322c9808a986920835224b498
(3) 0x5f1022a0441169543455caee5aa4a9ddaa780820bf10d6195a41c207ee2a90c5
(4) 0x7423cbfea5e05c707099b9fdb76e4e72ce5e2ec883c39e21063402d741b8c6c2f
(5) 0x882ed31a2602f7008d27806afa209d79e83da646ac4207c4d8b87c59a810d97d
(6) 0x2a15b3a3294123054ec35348fcb7ea7d746b0dd91504f9483c7176200a30cd78
(7) 0x6b03c17392013296fd6f2db6f81a62e60bcd8af887242d317706c2113a8351c
(8) 0x916705a1e17c80e7b1c4c7fe44d4aad250b75f8f48092286923a1bd546f09ba2
(9) 0x57e9ea4a37f22580c0ddefc329bd8379fedffa2fc63a885d4b3601a57796dc70

HD Wallet
=====
Mnemonic:      avocado coil exclude place kiwi fever fortune village chat episode page permit
Base HD Path:  m/44'/60'/0'/0/{account_index}

Default Gas Price
=====
2000000000

BlockGas Limit
=====
30000000

Call Gas Limit
=====
50000000

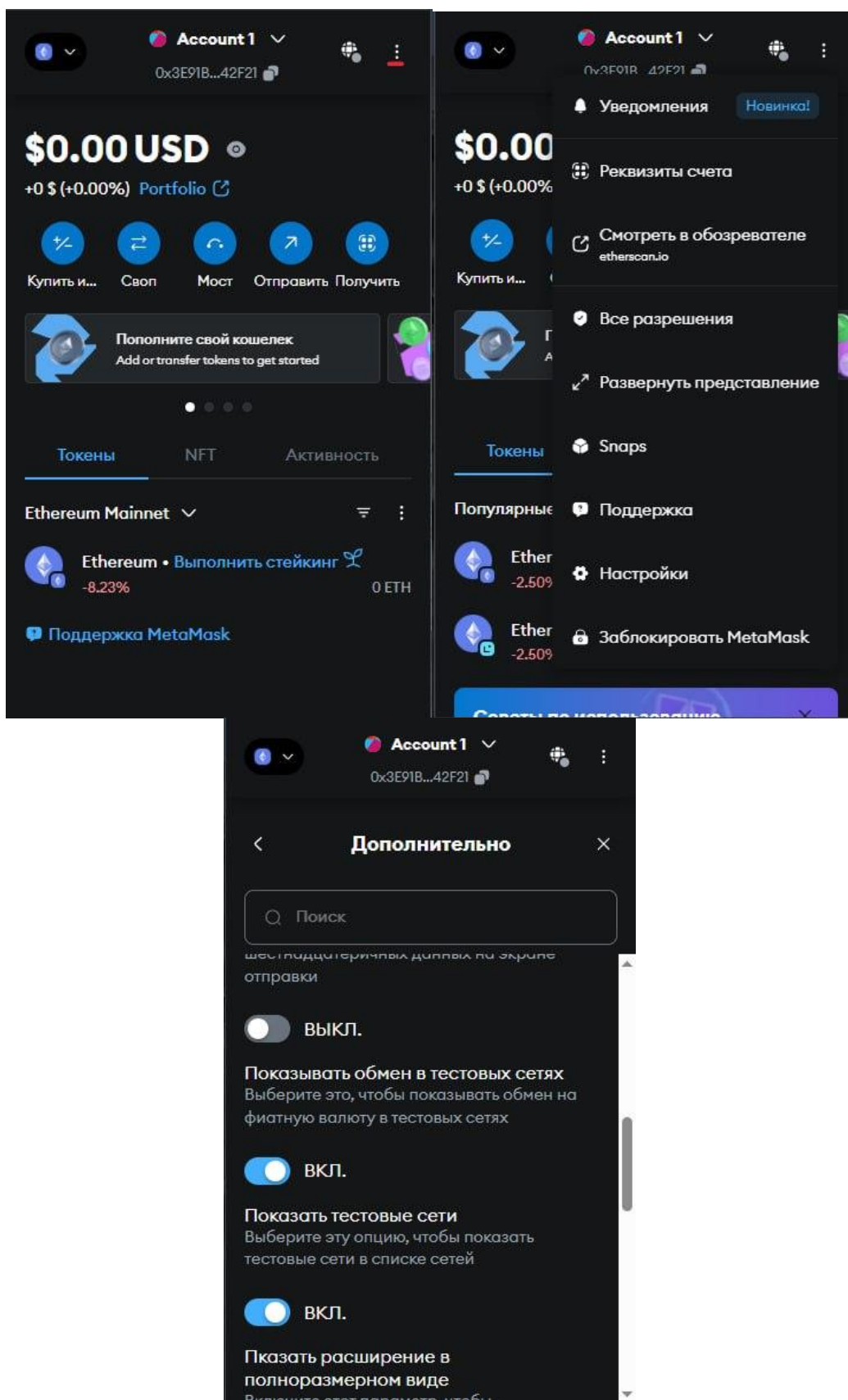
Chain
=====
Hardfork: shanghai
Id:      1337

RPC Listening on 127.0.0.1:8545

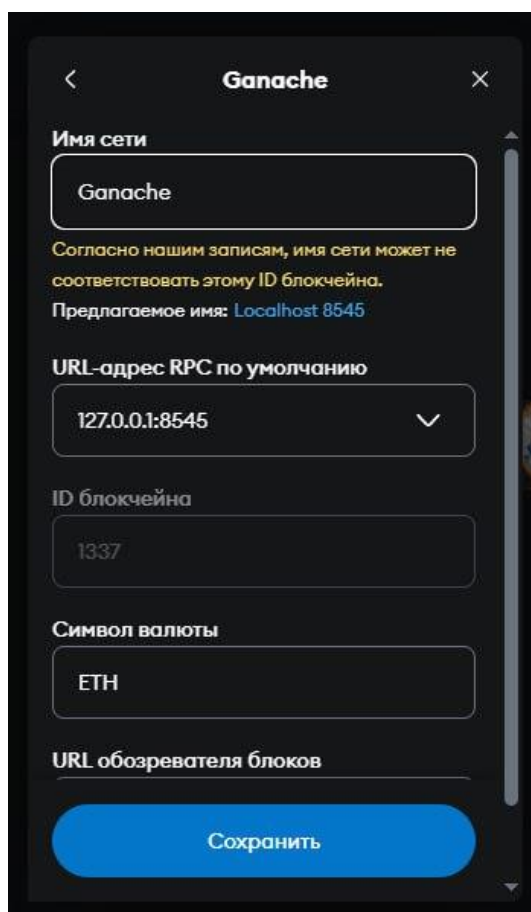
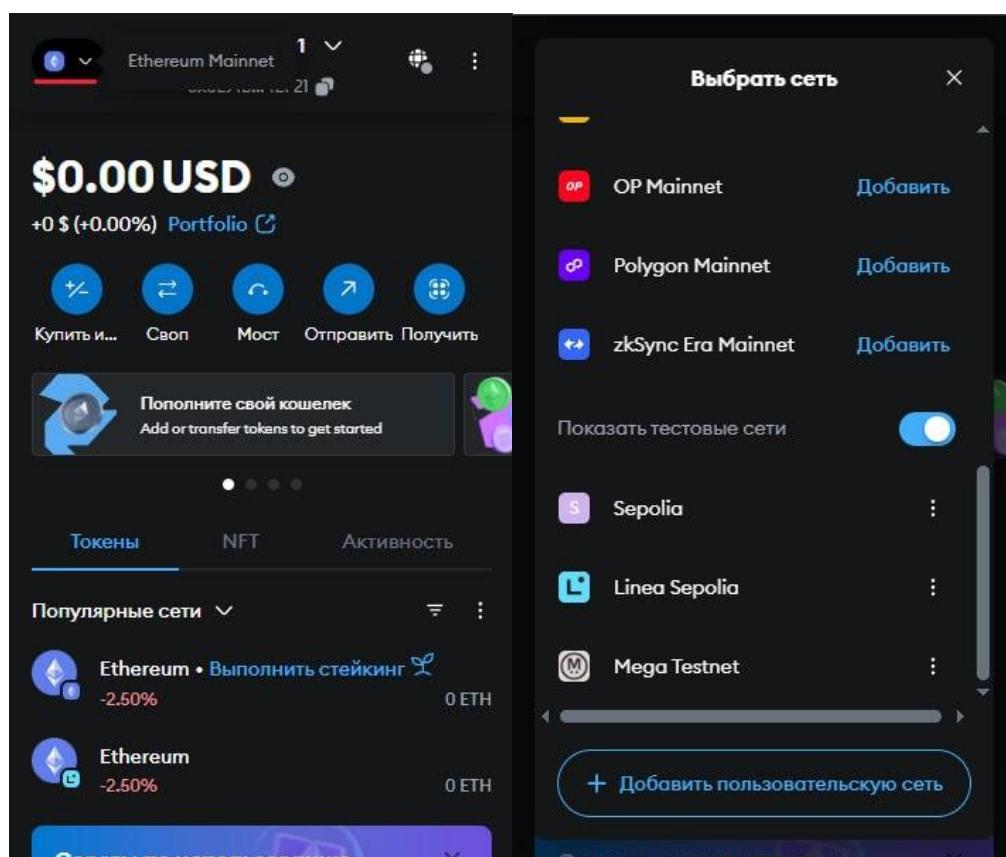
```

При помощи этой команды генерируются аккаунты с тестовой валютой и приватные ключи, которые понадобятся для добавления кошельков в Metamask.

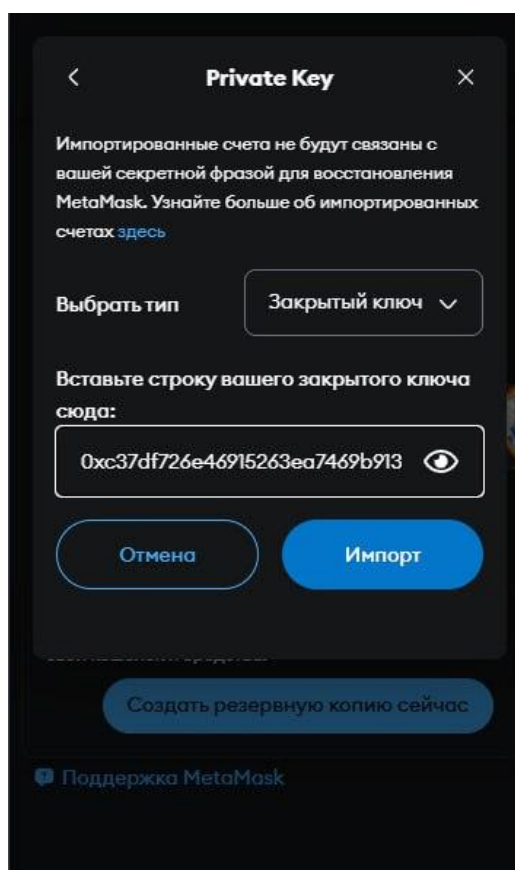
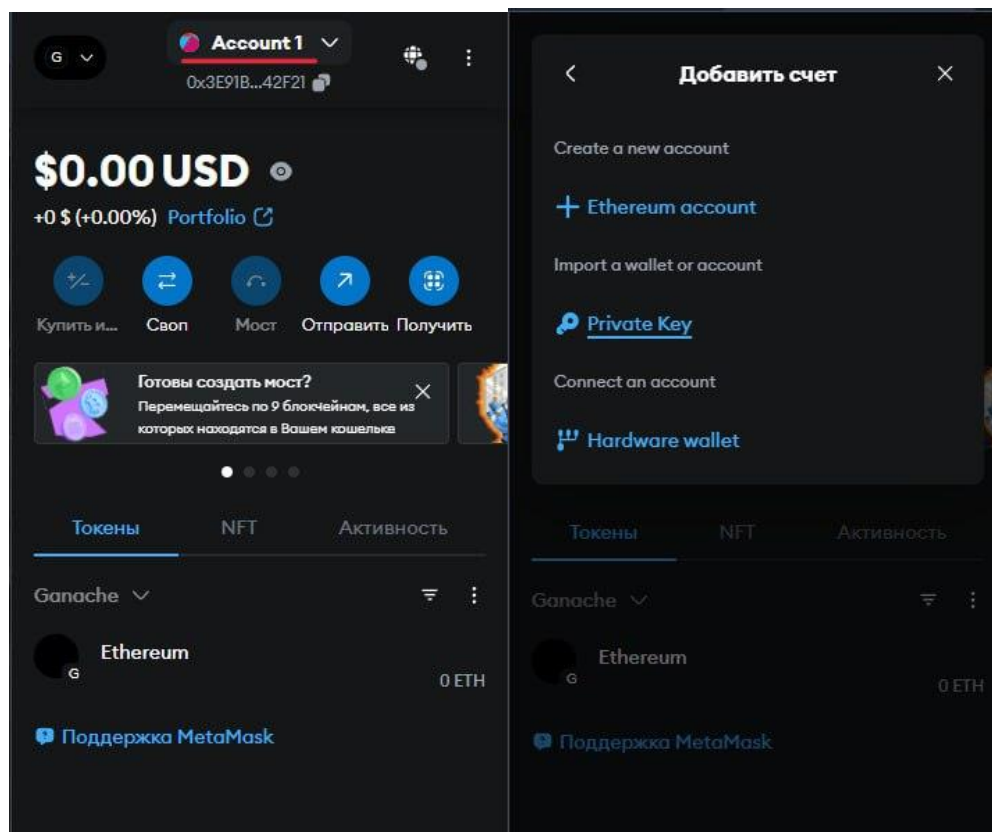
Далее необходимо добавить тестовую сеть Ganache в Metamask, для чего в настройках нужно поменять отображение тестовых сетей.



Затем нужно добавить тестовую сеть.



Из ранее выданных приватных ключей необходимо скопировать один из них и привязать к аккаунту в Metamask.



В папке contracts необходимо создать файл SmartRental.sol, содержащий код смарт-контракта.

Смарт-контракт — это программа, развернутая в блокчейне, которая автоматически исполняет заложенные в неё условия. После развертывания смарт-контракт нельзя изменить, а все его действия записываются в блокчейн, что обеспечивает прозрачность и надёжность.

Ниже приводится описание файла, в котором находится код смарт-контракта, описывающего один из игровых процессов многих RPG. Приведенный в качестве примера контракт позволяет сдавать в аренду виртуальные предметы с характеристиками («прочность», «зачарование») и управлять процессом аренды и возврата залога с учетом степени износа предмета. Он гарантирует справедливые расчёты между владельцем предмета и арендатором.

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.20;

contract SmartRental {

    // Структура, описывающая арендный предмет
    struct RentalItem {
        uint256 id;           // Уникальный идентификатор предмета
        string name;          // Название предмета
        uint256 durability;   // Прочность предмета
        string enchantment;   // Тип зачарования предмета
        address owner;        // Владелец предмета
        bool isRented;        // Флаг, находится ли предмет в аренде
    }

}
```

// Структура, содержащая информацию об аренде

```
struct RentalInfo {  
    address renter;      // Адрес арендатора  
    string renterName;   // Имя арендатора  
    uint256 rentStart;   // Время начала аренды (timestamp)  
    uint256 rentEnd;     // Время окончания аренды (timestamp)  
    uint256 deposit;     // Размер депозита  
    uint256 rentFee;     // Сумма платы за аренду  
    bool returned;       // Флаг возврата предмета  
    uint256 initialDurability; // Изначальная прочность  
    string initialEnchantment; // Изначальное зачарование  
}
```

// Хранилище предметов:

```
itemId => RentalItem
```

```
mapping(uint256 => RentalItem) public items;
```

// Хранилище информации об аренде:

```
itemId => RentalInfo
```

```
mapping(uint256 => RentalInfo) public rentals;
```

// Событие, срабатывающее при аренде предмета

```
event ItemRented(uint256 indexed itemId, address renter, string renterName);
```

// Событие, срабатывающее при возврате предмета

```
event ItemReturned(  
    uint256 indexed itemId,  
    uint256 finalDurability,  
    string finalEnchantment,  
    uint256 returnedDeposit  
);
```

```

    // Событие для отладки, показывает причину возврата депозита (или его
части)

    event DebugReturn(
        string reason,
        uint256 deposit,
        uint256 diffDur,
        uint8 initEnchant,
        uint8 finalEnchant,
        bool late
    );

    // адрес владельца для получения платы за аренду
    address public treasury = 0x80C238C355479AEaF62641D032559eB424a49a2C;
    address public bufferWallet = address(this); // контракт временно хранит
депозиты

    // Функция добавления нового предмета в систему
    function addItem(
        uint256 _id,
        string memory _name,
        uint256 _durability,
        string memory _enchantment
    ) public {
        items[_id] = RentalItem(_id, _name, _durability, _enchantment, msg.sender,
false);
    }

    // Функция аренды предмета
    function rentItem(
        uint256 _itemId,
        uint256 _durationInSeconds,
        string memory _renterName

```

```

) external payable {
    RentalItem storage item = items[_itemId];

    require(!item.isRented, "Already rented"); // проверка, что предмет свободен
    require(msg.value % 3 == 0, "Payment must be divisible by 3"); // плата
    должна делиться на 3

    uint256 rentFee = msg.value / 3;          // 1/3 — аренда
    uint256 deposit = rentFee * 2;           // 2/3 — депозит

    // Записываем информацию об аренде
    rentals[_itemId] = RentalInfo(
        msg.sender,
        _renterName,
        block.timestamp,
        block.timestamp + _durationInSeconds,
        deposit,
        rentFee,
        false,
        item.durability,
        item.enchantment
    );

    item.isRented = true;                    // помечаем предмет как арендованный
    payable(treasury).transfer(rentFee);    // переводим арендную плату
    владельцу

    emit ItemRented(_itemId, msg.sender, _renterName);
}

// Функция возврата предмета после аренды
function returnItem(

```

```

uint256 _itemId,
uint256 initialDurability,
string memory initialEnchantment,
uint256 finalDurability,
string memory finalEnchantment
) public {
    RentalItem storage item = items[_itemId];
    RentalInfo storage rental = rentals[_itemId];

    require(msg.sender == rental.renter, "Not renter"); // только арендатор
    может вернуть
    require(!rental.returned, "Already returned"); // проверка, что предмет
    ещё не возвращён
    // Обновление состояния
    item.isRented = false;
    item.durability = finalDurability;
    item.enchantment = finalEnchantment;
    rental.returned = true;

    uint256 returnedDeposit = rental.deposit;
    bool isLate = block.timestamp > rental.rentEnd; // проверка на просрочку

    uint256 diffDur = 0;
    if (initialDurability > finalDurability) {
        diffDur = initialDurability - finalDurability; // разница в прочности
    }

    uint8 initEnchant = getEnchantmentLevel(initialEnchantment); // уровень
    начального зачарования
    uint8 finalEnchant = getEnchantmentLevel(finalEnchantment); // уровень

```


финального зачарования

// Логика удержания депозита

```
if (isLate) {
    returnedDeposit = 0;
    emit DebugReturn("Late return - no deposit", rental.deposit, diffDur,
initEnchant, finalEnchant, true);
} else {
    if (finalEnchant < initEnchant) {
        if (diffDur >= 15) {
            returnedDeposit = 0;
            emit DebugReturn("Both downgraded - no deposit", rental.deposit,
diffDur, initEnchant, finalEnchant, false);
        } else {
            returnedDeposit = rental.deposit / 2;
            emit DebugReturn("Enchantment downgraded - half deposit",
rental.deposit, diffDur, initEnchant, finalEnchant, false);
        }
    } else if (diffDur >= 15) {
        returnedDeposit = rental.deposit / 2;
        emit DebugReturn("Durability dropped - half deposit", rental.deposit,
diffDur, initEnchant, finalEnchant, false);
    } else {
        emit DebugReturn("All good - full deposit", rental.deposit, diffDur,
initEnchant, finalEnchant, false);
    }
}
```

// Возврат оставшегося депозита арендатору

```
if (returnedDeposit > 0) {
    payable(rental.renter).transfer(returnedDeposit);
}
```

```

    }

    // Остаток депозита (если был удержан) переводится владельцу
    if (rental.deposit - returnedDeposit > 0) {
        payable(treasury).transfer(rental.deposit - returnedDeposit);
    }

    emit ItemReturned(_itemId, finalDurability, finalEnchantment,
returnedDeposit);
}

// Вспомогательная функция — переводит тип зачарования в числовой
уровень
function getEnchantmentLevel(string memory enchantment) internal pure returns
(uint8) {
    bytes32 hashVal = keccak256(abi.encodePacked(enchantment));
    if (hashVal == keccak256(abi.encodePacked("common"))) return 1;
    if (hashVal == keccak256(abi.encodePacked("rare"))) return 2;
    if (hashVal == keccak256(abi.encodePacked("epic"))) return 3;
    if (hashVal == keccak256(abi.encodePacked("legendary"))) return 4;
    return 0; // неизвестный тип
}

// Получить информацию о конкретном предмете
function getItem(uint256 _itemId) public view returns (RentalItem memory) {
    return items[_itemId];
}

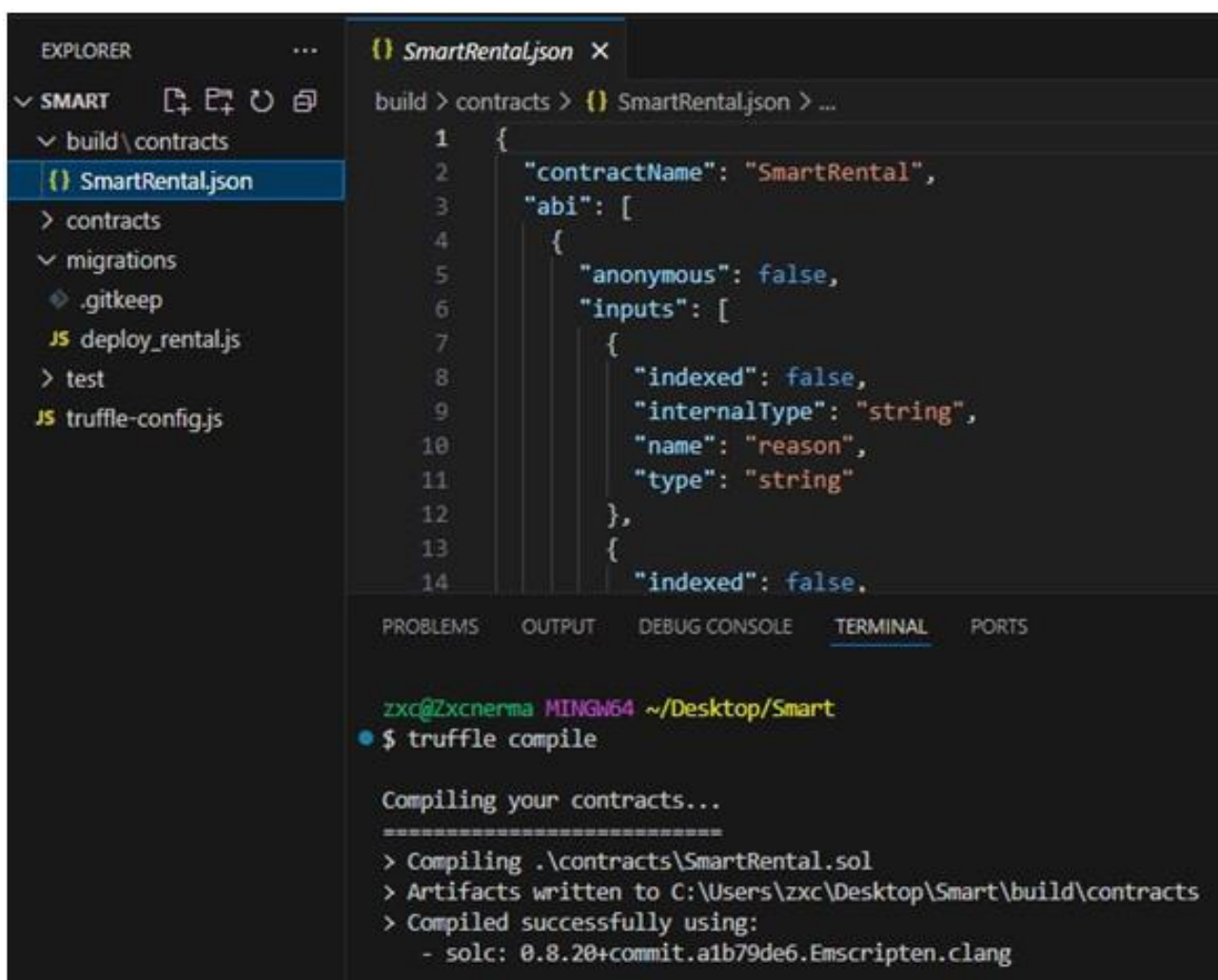
// Получить информацию об аренде конкретного предмета
function getRental(uint256 _itemId) public view returns (RentalInfo memory) {
    return rentals[_itemId];
}
}

```

Далее необходимо скомпилировать контракт при помощи Truffle, вследствие чего создается файл с ABI контракта.

ABI — это набор описаний функций и событий, с которыми можно взаимодействовать с контрактом снаружи (через Web3, ethers.js, другие dApp-интерфейсы).

truffle compile



```
EXPLORER
SMART
  build\contracts
    {} SmartContract.json
  > contracts
  > migrations
    .gitkeep
  JS deploy_rental.js
  > test
  JS truffle-config.js

{} SmartContract.json
build > contracts > {} SmartContract.json > ...
1  {
2    "contractName": "SmartContract",
3    "abi": [
4      {
5        "anonymous": false,
6        "inputs": [
7          {
8            "indexed": false,
9            "internalType": "string",
10           "name": "reason",
11           "type": "string"
12         },
13         {
14           "indexed": false,
```

```
zxc@Zxcnema MINGW64 ~/Desktop/Smart
$ truffle compile

Compiling your contracts...
=====
> Compiling .\contracts\SmartContract.sol
> Artifacts written to C:\Users\zxc\Desktop\Smart\build\contracts
> Compiled successfully using:
  - solc: 0.8.20+commit.a1b79de6.Emscripten.clang
```

Затем необходимо развернуть контракт в сети Ganache. Для этого используется команда **truffle migrate**, что позволяет управлять миграциями, то есть последовательными изменениями и обновлениями контрактов, сохраняя историю развёртываний.

truffle migrate

```
zxc@Zxcnerma MINGW64 ~/Desktop/Smart
$ truffle migrate

Compiling your contracts...
=====
> Compiling .\contracts\SmartRental.sol
> Artifacts written to C:\Users\zxc\Desktop\Smart\build\contracts
> Compiled successfully using:
  - solc: 0.8.20+commit.a1b79de6.Emscripten.clang

Starting migrations...
=====
> Network name:      'development'
> Network id:        1744212457800
> Block gas limit: 30000000 (0x1c9c380)

1_deploy_rental.js
=====

Deploying 'SmartRental'
-----
> transaction hash: 0xfb98107fcb2eeae5ba7f9d7cf5906f06e48145da67ce60e9d2a42660922fea62
> Blocks: 0
> contract address: 0x1FEF2Cc9cbbF10B8E5d770B54B00b981D70CD6c0
> block number: 2
> block timestamp: 1744213644
> account: 0x710a170e6E6Bb72F915282DDb991d257D9F1102D
> balance: 999.98488529159932201
> gas used: 2270437 (0x22a4e5)
> gas price: 3.28218027 gwei
> value sent: 0 ETH
> total cost: 0.00745198352567799 ETH

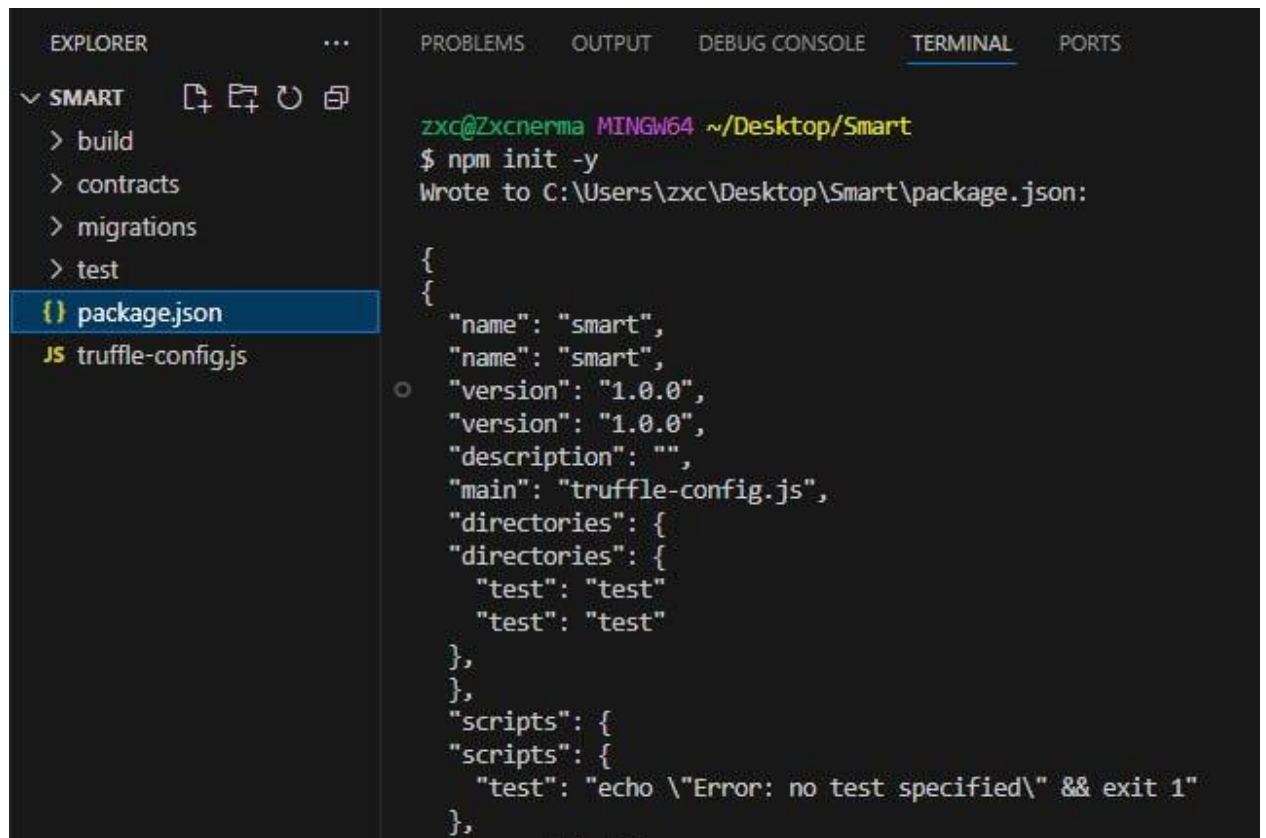
> Saving artifacts
-----
> Total cost: 0.00745198352567799 ETH

Summary
=====
> Total deployments: 1
> Final cost: 0.00745198352567799 ETH
```

Для frontend-части проекта следует настроить Node.js сервер, а также необходимо установить фреймворк express для обработки зависимостей JSON-файлов

npm init -y

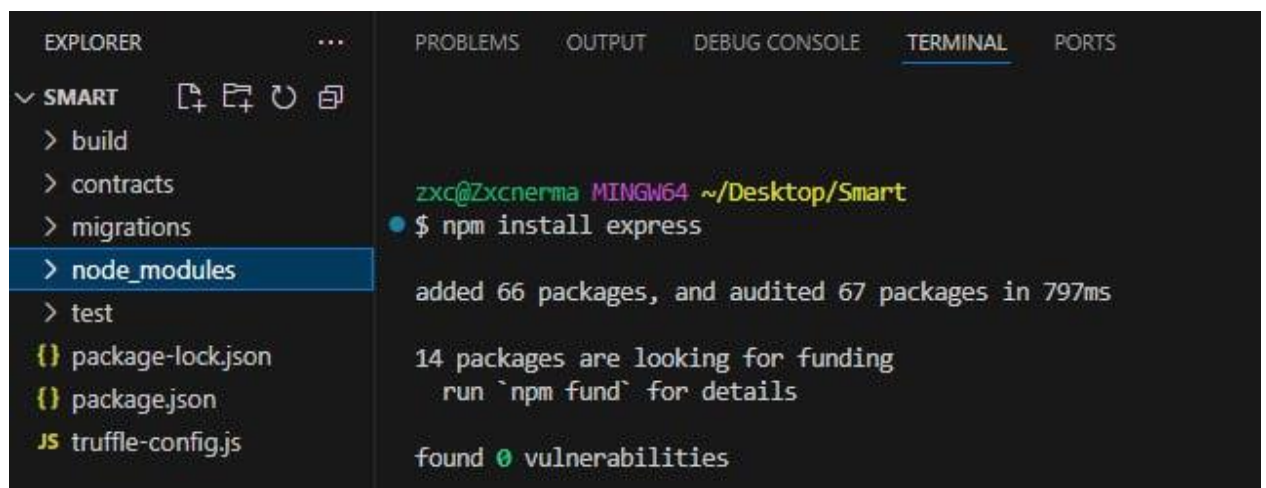
npm install express



The screenshot shows the VS Code interface with the Explorer on the left and the Terminal on the right. In the Explorer, the 'SMART' folder is expanded, showing files like 'build', 'contracts', 'migrations', 'test', 'package.json', and 'truffle-config.js'. The 'package.json' file is selected. The Terminal shows the command 'npm init -y' being executed, which has created a 'package.json' file in the project directory. The content of 'package.json' is displayed in the terminal output.

```
zxc@Zxcnerma MINGW64 ~/Desktop/Smart
$ npm init -y
Wrote to C:\Users\zxc\Desktop\Smart\package.json:

{
  "name": "smart",
  "version": "1.0.0",
  "description": "",
  "main": "truffle-config.js",
  "directories": {
    "test": "test"
  },
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  }
}
```



The screenshot shows the VS Code interface with the Explorer on the left and the Terminal on the right. In the Explorer, the 'node_modules' folder is selected. The Terminal shows the command 'npm install express' being executed. The output indicates that 66 packages were added and 67 packages were audited in 797ms. It also shows that 14 packages are looking for funding and that no vulnerabilities were found.

```
zxc@Zxcnerma MINGW64 ~/Desktop/Smart
$ npm install express

added 66 packages, and audited 67 packages in 797ms

14 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

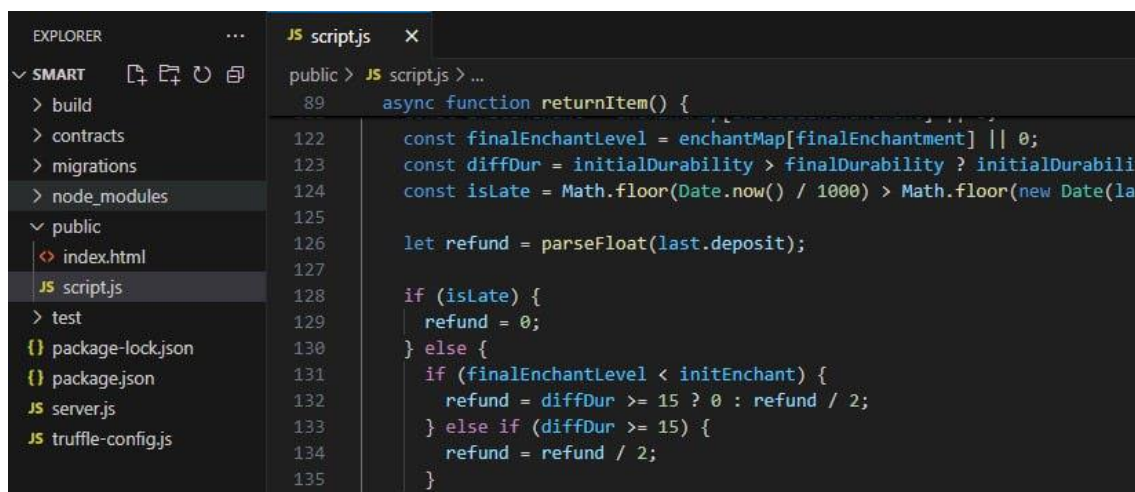
В корневой папке проекта необходимо создать файл **server.js**.



The screenshot shows the VS Code interface with the Explorer on the left and the Editor on the right. In the Explorer, the 'server.js' file is selected. The Editor shows the content of 'server.js', which includes imports for 'express', 'path', and 'fs', and code for setting up an Express app, using middleware, and loading a contract ABI.

```
JS server.js
JS server.js > ...
1 const express = require('express');
2 const path = require('path');
3 const fs = require('fs');
4 const app = express();
5 const PORT = 3000;
6
7 // Middleware для обработки JSON
8 app.use(express.json());
9 app.use(express.static(path.join(__dirname, 'public')));
10
11 // Загрузка ABI контракта
12 const contractABI = JSON.parse(fs.readFileSync('./build/contracts/SmartRental.json', 'utf8')).abi;
```

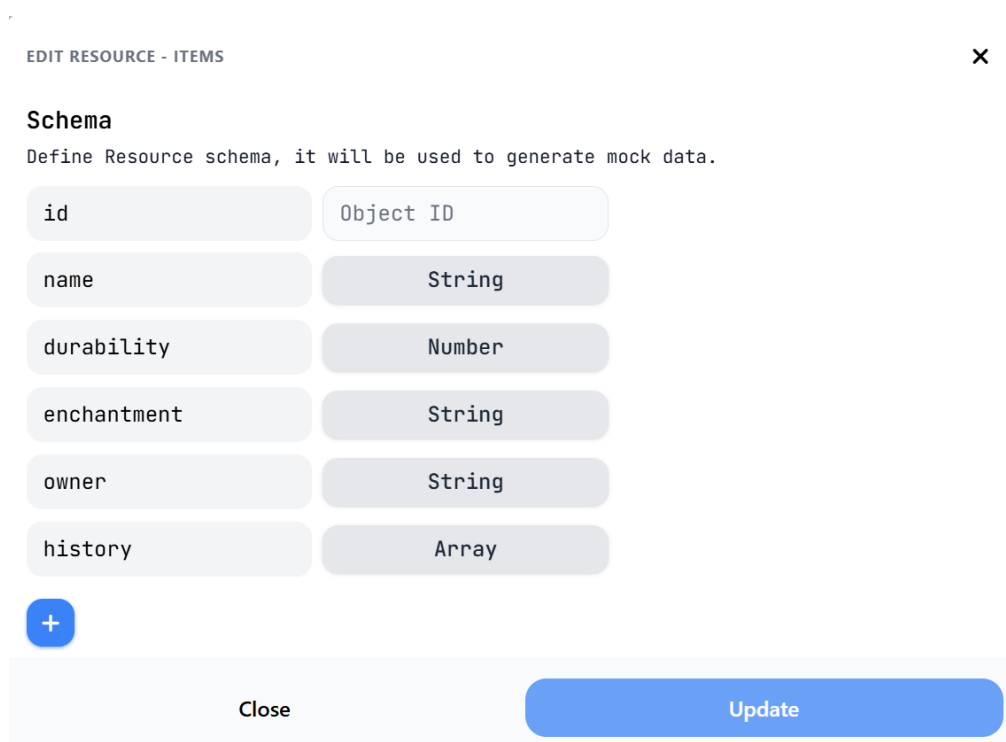
Там же нужно создать папку `public`, в которой будет находиться `html`-страница сайта и привязанный к ней код `javascript`.



```
89  async function returnItem() {
122  const finalEnchantLevel = enchantMap[finalEnchantment] || 0;
123  const diffDur = initialDurability > finalDurability ? initialDurability : finalDurability;
124  const isLate = Math.floor(Date.now() / 1000) > Math.floor(new Date(lastDeposit).getTime() / 1000);
125
126  let refund = parseFloat(last.deposit);
127
128  if (isLate) {
129    refund = 0;
130  } else {
131    if (finalEnchantLevel < initEnchant) {
132      refund = diffDur >= 15 ? 0 : refund / 2;
133    } else if (diffDur >= 15) {
134      refund = refund / 2;
135    }
136  }
```

В `MockAPI` хранятся данные о предметах смарт-контракта.

MockAPI — это онлайн-сервис для создания фейковых (тестовых) REST API. Он позволяет разработчикам симулировать поведение серверной части, не имея настоящей базы данных. `MockAPI` удобно использовать при фронтенд-разработке, тестировании и демонстрации работы с внешними API.



EDIT RESOURCE - ITEMS

Schema

Define Resource schema, it will be used to generate mock data.

id	Object ID
name	String
durability	Number
enchantment	String
owner	String
history	Array

+

Close Update

Ниже приводится описание script.js.

```
let web3, contract, accounts;

// Переменные для хранения экземпляра web3, контракта и списка аккаунтов

const mockApiUrl = "https://67d2ea018bca322cc268713f.mockapi.io/items";
// URL mockAPI, хранящего информацию о предметах и их истории

async function init() {
  // Инициализация Web3, получение аккаунтов и подключение к контракту
  web3 = new Web3(window.ethereum); // Создание экземпляра web3 через
  Metamask
  try {
    await window.ethereum.request({ method: "eth_requestAccounts" });
    // Запрос доступа к аккаунтам у пользователя
    accounts = await web3.eth.getAccounts(); // Получение списка аккаунтов

    const response = await fetch('/api/contract-info');
    // Получение ABI и адреса контракта с сервера
    const { abi, address } = await response.json();

    contract = new web3.eth.Contract(abi, address);
    // Создание экземпляра контракта
    console.log("Контракт инициализирован:", contract);
  } catch (error) {
    console.error("Ошибка:", error);
    alert("Ошибка подключения к контракту.");
  }
}
```

```

// Функция расчёта общей суммы аренды + депозита
function updateTotal() {
    const rent = parseFloat(document.getElementById("rentPrice").value || 0);
    const total = (rent * 3).toFixed(3); // Аренда + двойной депозит
    document.getElementById("totalPay").innerText = total; // Отображение суммы
на странице
}

// Загрузка информации о предмете по его ID
async function loadItem() {
    const id = document.getElementById("itemId").value;
    const res = await fetch(`${mockApiUrl}/${id}`); // Получение предмета по ID
    const item = await res.json();
    const last = item.history?.[item.history.length - 1]; // Последняя запись об аренде
(если есть)

    // Отображение информации о предмете на странице
    document.getElementById("itemInfo").innerHTML = `
        <h4>${item.name}</h4>
        <p>Прочность: ${last?.durabilityAfterUse ?? item.durability}</p>
        <p>Зачарование: ${last?.enchantment ?? item.enchantment}</p>
        <p>Владелец: ${item.owner}</p>
    `;
}

// Оформление аренды предмета
async function rentItem() {
    const id = document.getElementById("itemId").value;
    const duration = document.getElementById("duration").value;
    const nickname = document.getElementById("nickname").value;

```

```

const rentFeeETH = document.getElementById("rentPrice").value;
const rentFeeWei = web3.utils.toWei(rentFeeETH, "ether");
const totalWei = BigInt(rentFeeWei) * 3n; // Полная сумма (аренда + 2x
депозит)

const now = new Date().toISOString();
const endTime = new Date(Date.now() + duration * 1000).toISOString();

// Вызов метода контракта для аренды
await contract.methods.rentItem(id, duration, nickname).send({
  from: accounts[0],
  value: totalWei.toString()
});
// Получение предмета и добавление новой записи в его историю
const res = await fetch(`${mockApiUrl}/${id}`);
const item = await res.json();
item.history = item.history || [];
const lastEntry = item.history[item.history.length - 1] || {};
const initDurability = lastEntry.durabilityAfterUse || item.durability;
const initEnchantment = lastEntry.enchantment || item.enchantment;

item.history.push({
  renter: nickname,
  rentedAt: now,
  rentEnd: endTime,
  returnedAt: "",
  durabilityAfterUse: "",
  enchantment: "",
  initialDurability: initDurability,
  initialEnchantment: initEnchantment,

```

```

    rentFee: rentFeeETH,
    deposit: (parseFloat(rentFeeETH) * 2).toFixed(3)
  });

  // Сохраняем обновлённую историю аренды
  await fetch(`${mockApiUrl}/${id}`, {
    method: "PUT",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(item)
  });

  alert("Аренда оформлена и история записана!");
}
// Возврат предмета и расчёт возврата депозита
async function returnItem() {
  const id = document.getElementById("itemId").value;
  const res = await fetch(`${mockApiUrl}/${id}`);
  const item = await res.json();
  const last = item.history?.[item.history.length - 1];

  if (!last) {
    alert("История не найдена!");
    return;
  }

  const initialDurability = parseInt(last.initialDurability);
  const initialEnchantment = last.initialEnchantment;

  const finalDurability = Math.floor(Math.random() * 30) + 70; // Случайное
  снижение прочности

```

```

const enchantments = ["common", "rare", "epic", "legendary"];
const finalEnchantment = enchantments[Math.floor(Math.random() *
enchantments.length)]; // Случайное зачарование

// Вызов метода контракта для возврата предмета
await contract.methods
    .returnItem(id, initialDurability, initialEnchantment, finalDurability,
finalEnchantment)
    .send({ from: accounts[0] });

// Обновление истории
const now = new Date().toISOString();
last.returnedAt = now;
last.durabilityAfterUse = finalDurability;
last.enchantment = finalEnchantment;

// Расчёт возврата депозита по тем же правилам, что в контракте
const enchantMap = { "common": 1, "rare": 2, "epic": 3, "legendary": 4 };
const initEnchant = enchantMap[initialEnchantment] || 0;
const finalEnchantLevel = enchantMap[finalEnchantment] || 0;
const diffDur = initialDurability > finalDurability ? initialDurability -
finalDurability : 0;
const isLate = Math.floor(Date.now() / 1000) > Math.floor(new
Date(last.rentEnd).getTime() / 1000);

let refund = parseFloat(last.deposit);
if (isLate) {
    refund = 0;
} else {
    if (finalEnchantLevel < initEnchant) {

```

```

    refund = diffDur >= 15 ? 0 : refund / 2;
  } else if (diffDur >= 15) {
    refund = refund / 2;
  }
}

last.refund = refund.toFixed(3) + " ETH"; // Итоговая сумма возврата
await fetch(`${mockApiUrl}/${id}`, {
  method: "PUT",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(item)
});
alert("Предмет возвращён, история обновлена.");
}
// Загрузка истории аренды предмета
async function loadHistory() {
  const id = document.getElementById("itemId").value;

  const res = await fetch(`${mockApiUrl}/${id}`);

  const item = await res.json();

  const hist = item.history || [];

  const html = hist.map(entry => `
    <div style="border-bottom:1px solid #ccc; margin-bottom:10px; padding-
bottom:10px;">
      <strong>Ник:</strong> ${entry.renter || "-"}<br>
      <strong>Аренда:</strong> ${entry.rentedAt} ? new
Date(entry.rentedAt).toLocaleString("ru-RU", { timeZone: "Europe/Moscow" }) : "-
"><br>

```



```

        <strong>Возврат:</strong>      ${entry.returnedAt      ?      new
Date(entry.returnedAt).toLocaleString("ru-RU", { timeZone: "Europe/Moscow" }) :
"Ещё не возвращено"}<br>
        <strong>Прочность:</strong>    ${entry.initialDurability    ||    "-"}    →
${entry.durabilityAfterUse || "-"}<br>
        <strong>Зачарование:</strong>   ${entry.initialEnchantment   ||   "-"}   →
${entry.enchantment || "-"}<br>
        <strong>Возврат депозита:</strong> ${entry.refund || "-"}
    </div>
    `).join("");

    document.getElementById("history").innerHTML = html;
}

```

Далее следует установить зависимости между файлами и запустить сервер.

npm install express fs

node server.js

```

zxc@Zxcnerma MINGW64 ~/Desktop/Smart
● $ npm install express fs

added 1 package, and audited 68 packages in 705ms

14 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

zxc@Zxcnerma MINGW64 ~/Desktop/Smart
○ $ node server.js
Сервер запущен на http://localhost:3000

```

Начальная страница сайта, с помощью которого стороны взаимодействуют со смарт-контрактом.

Загрузка предмета

Загрузить

Арендовать предмет

Итого к оплате: 0.003 ETH

Арендовать

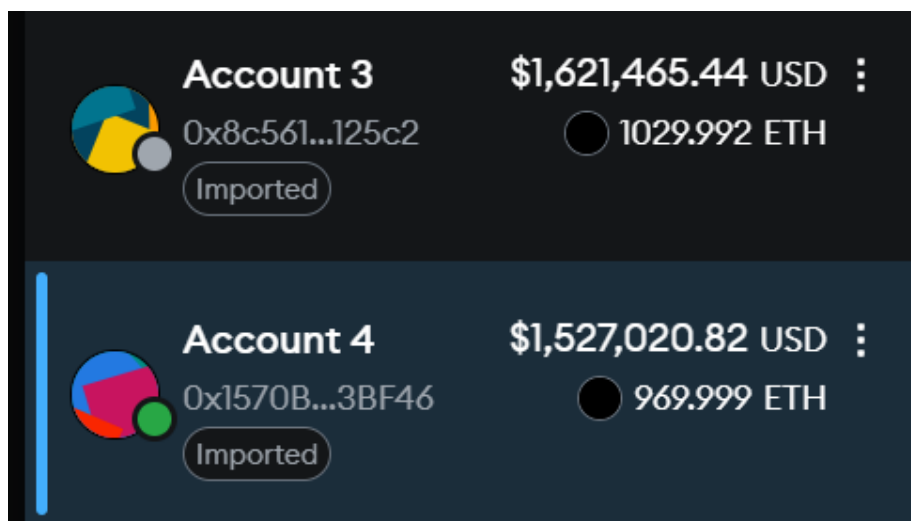
Вернуть предмет

Вернуть

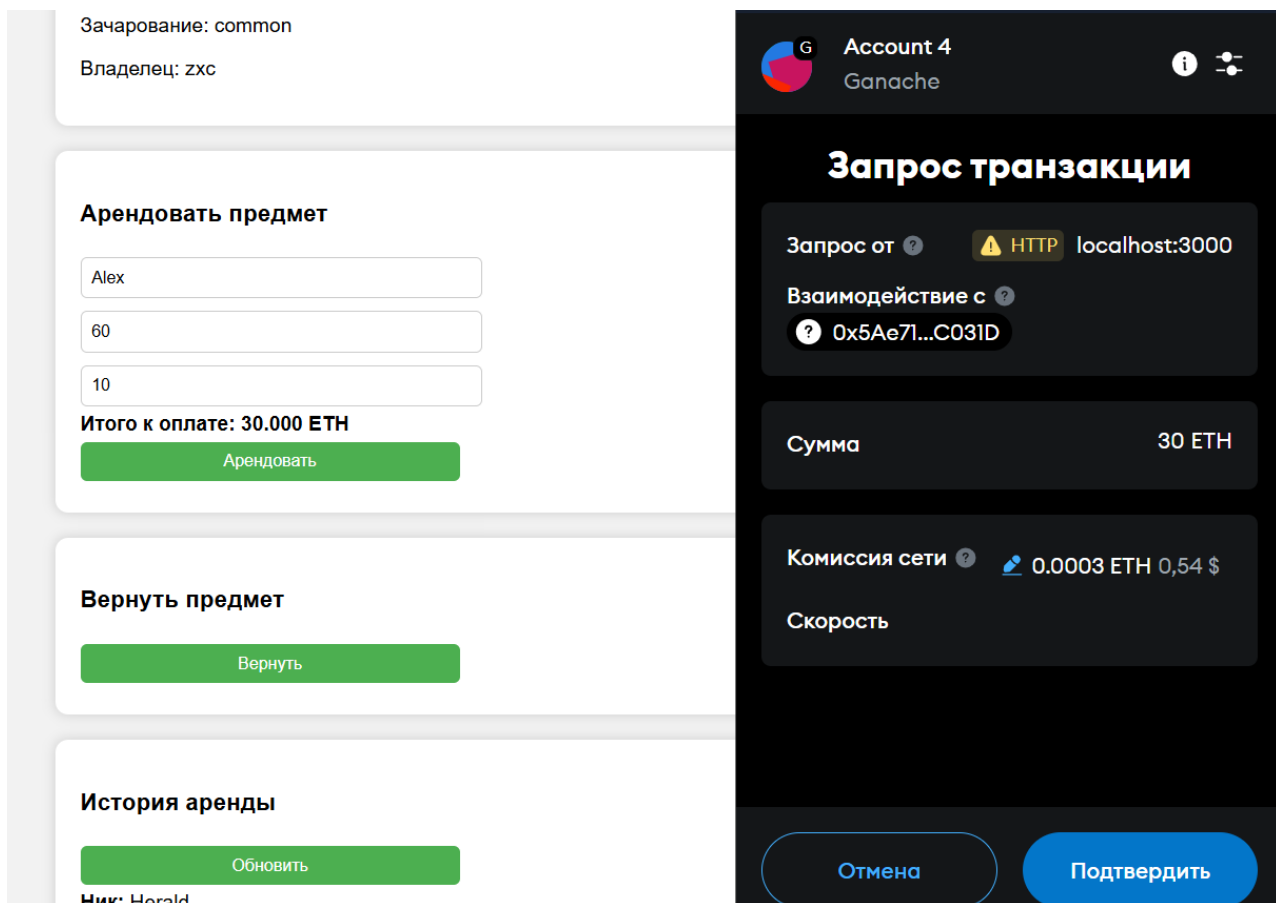
История аренды

Обновить

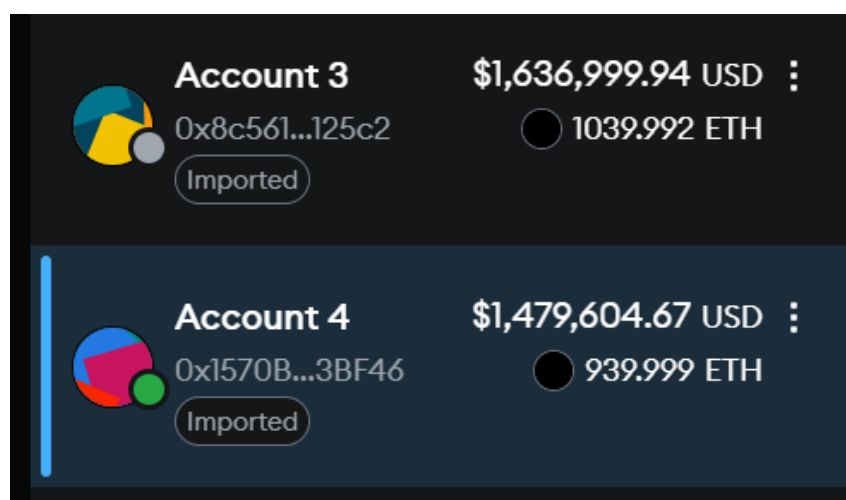
Баланс кошелька владельца и арендатора до аренды.



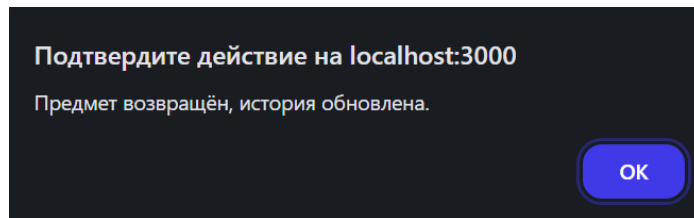
После ввода данных об арендаторе, времени аренды и сумме оплаты, нажимая кнопку «Арендовать», в расширении Metamask появляется запрос транзакции.



Баланс кошельков во время аренды.



После нажатия на кнопку «Вернуть» и подтверждения транзакции, выводится сообщение о возврате предмета.



Констатируется факт завершенной аренды с записью истории, залог возвращается на кошелек арендатора.

